

고객을 세그멘테이션하자 [프로젝트] - 조수경

고객을 세그멘테이션하자 [프로젝트] - 조수경 v1 (1)

11-2. 데이터 불러오기

데이터 살펴보기

- 테이블에 있는 10개의 행만 출력하기

```
# [[YOUR QUERY]]
```

```
SELECT *  
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`  
LIMIT 10
```

[결과 이미지를 넣어주세요]

쿼리 결과

+ 대화 만들기

결과 저장

다음에서 열기

작업 정보

결과

시각화

JSON

실행 세부정보

실행 그래프

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	536365	85123A	WHITE HANGING HEART T-LIGH...	6	2010-12-01 08:26:00 UTC	2.55	17850	United Kingdom
2	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
3	536365	84406B	CREAM CUPID HEARTS COAT H...	8	2010-12-01 08:26:00 UTC	2.75	17850	United Kingdom
4	536365	84029G	KNITTED UNION FLAG HOT WA...	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
5	536365	84029E	RED WOOLLY HOTTIE WHITE H...	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
6	536365	22752	SET 7 BABUSHKA NESTING BO...	2	2010-12-01 08:26:00 UTC	7.65	17850	United Kingdom
7	536365	21730	GLASS STAR FROSTED T-LIGHT ...	6	2010-12-01 08:26:00 UTC	4.25	17850	United Kingdom
8	536366	22633	HAND WARMER UNION JACK	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
9	536366	22632	HAND WARMER RED POLKA DOT	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
10	536367	84879	ASSORTED COLOUR BIRD ORN...	32	2010-12-01 08:34:00 UTC	1.69	13047	United Kingdom

- 전체 데이터는 몇 행으로 구성되어 있는지 확인하기

```
# [[YOUR QUERY]]
```

```
SELECT COUNT(*) AS total_rows  
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

쿼리 결과

작업 정보	결과	시
행	total_rows	
1	541909	

데이터 수 세기

- COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기

```
# [[YOUR QUERY]]

SELECT
  COUNT(InvoiceNo) AS InvoiceNo_count,
  COUNT(StockCode) AS StockCode_count,
  COUNT(Description) AS Description_count,
  COUNT(Quantity) AS Quantity_count,
  COUNT(InvoiceDate) AS InvoiceDate_count,
  COUNT(UnitPrice) AS UnitPrice_count,
  COUNT(CustomerID) AS CustomerID_count,
  COUNT(Country) AS Country_count
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

행	InvoiceNo_count	StockCode_count	Description_count	Quantity_count	InvoiceDate_count	UnitPrice_count	CustomerID_count	Country_count
1	541909	541909	540455	541909	541909	541909	406829	541909

11-4. 데이터 전처리 방법(1): 결측치 제거

컬럼 별 누락된 값의 비율 계산

- 각 컬럼 별 누락된 값의 비율을 계산
 - 각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기

```
# [[YOUR QUERY]]

SELECT
  'InvoiceNo' AS column_name,
  ROUND(SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) / COUNT(*)
    * 100, 2) AS missing_percentage
```

```

FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`

UNION ALL

SELECT
    'StockCode' AS column_name,
    ROUND(SUM(CASE WHEN StockCode IS NULL THEN 1 ELSE 0 END) / COUNT(*)
* 100, 2) AS missing_percentage
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`

UNION ALL

SELECT
    'Description' AS column_name,
    ROUND(SUM(CASE WHEN Description IS NULL THEN 1 ELSE 0 END) / COUNT
(*) * 100, 2) AS missing_percentage
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`

UNION ALL

SELECT
    'Quantity' AS column_name,
    ROUND(SUM(CASE WHEN Quantity IS NULL THEN 1 ELSE 0 END) / COUNT(*)
* 100, 2) AS missing_percentage
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`

UNION ALL

SELECT
    'InvoiceDate' AS column_name,
    ROUND(SUM(CASE WHEN InvoiceDate IS NULL THEN 1 ELSE 0 END) / COUNT
(*) * 100, 2) AS missing_percentage
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`

UNION ALL

SELECT
    'UnitPrice' AS column_name,
    ROUND(SUM(CASE WHEN UnitPrice IS NULL THEN 1 ELSE 0 END) / COUNT(*)
* 100, 2) AS missing_percentage
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`

UNION ALL

SELECT
    'CustomerID' AS column_name,
    ROUND(SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 END) / COUNT
(*) * 100, 2) AS missing_percentage

```

```
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`

UNION ALL

SELECT
  'Country' AS column_name,
  ROUND(SUM(CASE WHEN Country IS NULL THEN 1 ELSE 0 END) / COUNT(*) *
100, 2) AS missing_percentage
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

행	column_name	missing_percenta...
1	InvoiceNo	0.0
2	StockCode	0.0
3	Description	0.27
4	Quantity	0.0
5	InvoiceDate	0.0
6	UnitPrice	0.0
7	CustomerID	24.93
8	Country	0.0

결측치 처리 전략

- `StockCode = '85123A'` 의 `Description` 을 추출하는 쿼리문을 작성하기

```
SELECT DISTINCT Description
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
WHERE StockCode = '85123A'
ORDER BY Description;
```

[결과 이미지를 넣어주세요]

행	Description
1	?
2	CREAM HANGING HEART T-LIG...
3	WHITE HANGING HEART T-LIGH...
4	wrongly marked carton 22804

결측치 처리

- DELETE 구문을 사용하며, WHERE 절을 통해 데이터를 제거할 조건을 제시

```
DELETE FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`  
WHERE CustomerID IS NULL;
```

[결과 이미지를 넣어주세요]

작업 정보 **결과** 실행 세부정보 실행 그래프

i 이 문으로 data의 행 135,080개가 삭제되었습니다.

11-5. 데이터 전처리(2): 중복값 처리

중복값 확인

- 중복된 행의 수를 세어보기
 - 8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터를 세어보기

```
SELECT COUNT(*) AS duplicate_rows  
FROM (  
  SELECT  
    InvoiceNo,  
    StockCode,  
    Description,  
    Quantity,  
    InvoiceDate,  
    UnitPrice,  
    CustomerID,  
    Country,  
    COUNT(*) AS cnt  
  FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`  
  GROUP BY  
    InvoiceNo,  
    StockCode,  
    Description,  
    Quantity,  
    InvoiceDate,  
    UnitPrice,  
    CustomerID,  
    Country  
  HAVING COUNT(*) > 1  
);
```

[결과 이미지를 넣어주세요]

행	duplicate_rows
1	4837

중복값 처리

- 중복값을 제거하는 쿼리문 작성하기

- CREATE OR REPLACE TABLE 구문을 활용하여 모든 컬럼(*)을 DISTINCT 한 데이터로 업데이트

```
CREATE OR REPLACE TABLE `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data` AS
SELECT DISTINCT *
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

 이 문으로 이름이 data인 테이블이 교체되었습니다.

11-6. 데이터 전처리(3): 오류값 처리

InvoiceNo 살펴보기

- 고유(unique)한 InvoiceNo의 개수를 출력하기

```
SELECT COUNT(DISTINCT InvoiceNo) AS unique_invoice_count
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

행	unique_invoice_c...
1	22190

- 고유한 InvoiceNo를 앞에서부터 100개를 출력하기

```
SELECT DISTINCT InvoiceNo
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
LIMIT 100;
```

[결과 이미지를 넣어주세요]

행	InvoiceNo	
1	541431	
2	C541433	
3	537626	
4	542237	
5	549222	
6	556201	
7	562032	
8	573511	
9	581180	
10	539318	
90	567928	
91	571187	
92	577180	
93	C572532	
94	563100	
95	570681	
96	570725	
97	574694	
98	580638	
99	C565050	
100	539840	

- **InvoiceNo** 가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)

```
SELECT *
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
WHERE InvoiceNo LIKE 'C%'
LIMIT 100;
```

[결과 이미지를 넣어주세요]

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	C541433	23166	MEDIUM CERAMIC TOP STORA...	-74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom
2	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	280.05	12352	Norway
3	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	183.75	12352	Norway
4	C545330	M	Manual	-1	2011-03-01 15:49:00 UTC	376.5	12352	Norway
5	C547388	21914	BLUE HARMONICA IN BOX	-12	2011-03-22 16:07:00 UTC	1.25	12352	Norway
6	C547388	22645	CERAMIC HEART FAIRY CAKE M...	-12	2011-03-22 16:07:00 UTC	1.45	12352	Norway
7	C547388	22784	LANTERN CREAM GAZEBO	-3	2011-03-22 16:07:00 UTC	4.95	12352	Norway
8	C547388	84050	PINK HEART SHAPE EGG FRYIN...	-12	2011-03-22 16:07:00 UTC	1.65	12352	Norway
9	C547388	22413	METAL SIGN TAKE IT OR LEAVE...	-6	2011-03-22 16:07:00 UTC	2.95	12352	Norway
10	C547388	37448	CERAMIC CAKE DESIGN SPOTT...	-12	2011-03-22 16:07:00 UTC	1.49	12352	Norway
11	C547388	22701	PINK DOG BOWL	-6	2011-03-22 16:07:00 UTC	2.95	12352	Norway
12	C549955	22666	RECIPE BOX PANTRY YELLOW D...	-2	2011-04-13 13:38:00 UTC	2.95	12359	Cyprus
13	C549955	22839	3 TIER CAKE TIN GREEN AND C...	-2	2011-04-13 13:38:00 UTC	14.95	12359	Cyprus
14	C580165	22826	LOVE SEAT ANTIQUE WHITE ME...	-1	2011-12-02 11:21:00 UTC	42.5	12359	Cyprus
15	C580165	23245	SET OF 3 REGENCY CAKE TINS	-2	2011-12-02 11:21:00 UTC	4.95	12359	Cyprus

85	C560540	23242	TREASURE TIN BUFFALO BILL	-1	2011-07-19 12:26:00 UTC	2.08	12415	Australia
86	C560540	21503	TOYBOX WRAP	-1	2011-07-19 12:26:00 UTC	0.42	12415	Australia
87	C560540	22973	CHILDREN'S CIRCUS PARADE M...	-1	2011-07-19 12:26:00 UTC	1.65	12415	Australia
88	C560540	23236	DOILEY STORAGE TIN	-1	2011-07-19 12:26:00 UTC	2.89	12415	Australia
89	C560540	20725	LUNCH BAG RED RETROSPOT	-1	2011-07-19 12:26:00 UTC	1.65	12415	Australia
90	C560540	21917	SET 12 KIDS WHITE CHALK STI...	-1	2011-07-19 12:26:00 UTC	0.42	12415	Australia
91	C560540	23174	REGENCY SUGAR BOWL GREEN	-1	2011-07-19 12:26:00 UTC	4.15	12415	Australia
92	C560540	21210	SET OF 72 RETROSPOT PAPER ...	-1	2011-07-19 12:26:00 UTC	1.45	12415	Australia
93	C560540	23240	SET OF 4 KNICK KNACK TINS D...	-1	2011-07-19 12:26:00 UTC	4.15	12415	Australia
94	C560540	23206	LUNCH BAG APPLE DESIGN	-1	2011-07-19 12:26:00 UTC	1.65	12415	Australia
95	C560540	23166	MEDIUM CERAMIC TOP STORA...	-1	2011-07-19 12:26:00 UTC	1.25	12415	Australia
96	C560540	23292	SPACEBOY CHILDRENS CUP	-1	2011-07-19 12:26:00 UTC	1.25	12415	Australia
97	C560540	23244	ROUND STORAGE TIN VINTAGE ...	-1	2011-07-19 12:26:00 UTC	1.95	12415	Australia
98	C560540	85099B	JUMBO BAG RED RETROSPOT	-1	2011-07-19 12:26:00 UTC	2.08	12415	Australia
99	C560540	20979	36 PENCILS TUBE RED RETROS...	-1	2011-07-19 12:26:00 UTC	1.25	12415	Australia
100	C560540	22150	3 STRIPEY MICE FELTCRAFT	-1	2011-07-19 12:26:00 UTC	1.95	12415	Australia

- 구매 건 상태가 **Canceled** 인 데이터의 비율(%) - 소수점 첫번째 자리까지

```
SELECT
  ROUND(SUM(CASE WHEN InvoiceNo LIKE 'C%' THEN 1 ELSE 0 END)
    / COUNT(*) * 100, 1) AS canceled_percentage
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

행	canceled_percent...
1	2.2

StockCode 살펴보기

- 고유한 **StockCode** 의 개수를 출력하기

```
SELECT COUNT(DISTINCT StockCode)AS unique_stockcode_count
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

행	unique_stockcod...
1	3684

- 어떤 제품이 가장 많이 판매되었는지 보기 위하여 **StockCode** 별 등장 빈도를 출력하기
 - 상위 10개의 제품들을 출력하기


```
SELECT StockCode, COUNT(*) AS sell_cnt
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
GROUP BY StockCode
ORDER BY sell_cnt DESC
LIMIT 10;
```

[결과 이미지를 넣어주세요]

행	StockCode	sell_cnt
1	85123A	2065
2	22423	1894
3	85099B	1659
4	47566	1409
5	84879	1405
6	20725	1346
7	22720	1224
8	POST	1196
9	22197	1110
10	23203	1108

- **StockCode**의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고
 - 숫자가 0~1개인 값들에는 어떤 코드들이 들어가 있는지 출력하기

#StockCode의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세기

```
WITH UniqueStockCodes AS (
  SELECT DISTINCT StockCode
  FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
)
SELECT
  LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count,
  COUNT(*) AS stock_cnt
FROM UniqueStockCodes
GROUP BY number_count
```

#숫자가 0~1개인 값들에는 어떤 코드들이 들어가 있는지 출력하기

```
SELECT DISTINCT StockCode, number_count
```

```

FROM (
    SELECT StockCode,
           LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', ''))
    AS number_count
    FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
)
WHERE number_count <= 1;

```

[결과 이미지를 넣어주세요]

행	number_count	stock_cnt
1	5	3676
2	0	7
3	1	1

행	StockCode	number_count
1	POST	0
2	M	0
3	C2	1
4	D	0
5	BANK CHARGES	0
6	PADS	0
7	DOT	0
8	CRUK	0

- **StockCode**의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고
 - 숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트인지 구하기 (소수점 두 번째 자리까지)

```

SELECT
ROUND(
SUM(CASE
WHEN LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) <= 1
THEN 1 ELSE 0
END)
/ COUNT(*) * 100,
2) AS percentage
FROM project-1fdc5a34-459d-4c61-a1b.modulabs_project.data;
SELECT DISTINCT StockCode, number_count
FROM (
    SELECT StockCode,
           LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', ''))
    AS number_count

```

```
FROM project_name.modulabs_project.data
)
WHERE # [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

행	percentage
1	0.48

• 제품과 관련되지 않은 거래 기록을 제거하기

```
DELETE FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
WHERE StockCode IN (
  SELECT DISTINCT StockCode
  FROM (
    SELECT StockCode,
      LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]',
        '')) AS number_count
    FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
  )
  WHERE number_count <= 1
);
```

[결과 이미지를 넣어주세요]

i 이 문으로 data의 행 1,915개가 삭제되었습니다.

Description 살펴보기

• 고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기

```
SELECT Description, COUNT(*) AS description_cnt
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
GROUP BY Description
ORDER BY description_cnt DESC
LIMIT 30;
```

[결과 이미지를 넣어주세요]

행	Description	description_cnt
1	WHITE HANGING HEART T-LIGH...	2058
2	REGENCY CAKESTAND 3 TIER	1894
3	JUMBO BAG RED RETROSPOT	1659
4	PARTY BUNTING	1409
5	ASSORTED COLOUR BIRD ORN...	1405
6	LUNCH BAG RED RETROSPOT	1345
7	SET OF 3 CAKE TINS PANTRY D...	1224
8	LUNCH BAG BLACK SKULL.	1099
9	PACK OF 72 RETROSPOT CAKE ...	1062
10	SPOTTY BUNTING	1026
11	PAPER CHAIN KIT 50'S CHRIST...	1013
12	LUNCH BAG SPACEBOY DESIGN	1006
13	LUNCH BAG CARS BLUE	1000
14	HEART OF WICKER SMALL	990
15	NATURAL SLATE HEART CHALK...	989

16	JAM MAKING SET WITH JARS	966
17	LUNCH BAG PINK POLKADOT	961
18	LUNCH BAG SUKI DESIGN	932
19	ALARM CLOCK BAKELIKE RED	917
20	REX CASH+CARRY JUMBO SHO...	900
21	WOODEN PICTURE FRAME WHI...	900
22	JUMBO BAG PINK POLKADOT	897
23	SET OF 4 PANTRY JELLY MOUL...	890
24	LUNCH BAG APPLE DESIGN	890
25	BAKING SET 9 PIECE RETROSP...	885
26	JAM MAKING SET PRINTED	883
27	RECIPE BOX PANTRY YELLOW D...	883
28	LUNCH BAG WOODLAND	850
29	ROSES REGENCY TEACUP AND ...	844
30	VICTORIAN GLASS HANGING T...	843

• 서비스 관련 정보를 포함하는 행들을 제거하기

```
DELETE
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
WHERE Description IN ('Next Day Carriage', 'High Resolution Image');
```

[결과 이미지를 넣어주세요]

i 이 문으로 data의 행 83개가 삭제되었습니다.

• 대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기

```
CREATE OR REPLACE TABLE `project-1fdc5a34-459d-4c61-a1b.modulabs_projec
t.data` AS
SELECT
  * EXCEPT (Description),
  UPPER(Description) AS Description
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

i 이 문으로 이름이 data인 테이블이 교체되었습니다.

UnitPrice 살펴보기

- UnitPrice 의 최솟값, 최댓값, 평균을 구하기

```
SELECT
  MIN(UnitPrice) AS min_price,
  MAX(UnitPrice) AS max_price,
  AVG(UnitPrice) AS avg_price
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

행	min_price	max_price	avg_price
1	0.0	649.5	2.904956757406...

- 단가가 0원인 거래의 개수, 구매 수량(Quantity)의 최솟값, 최댓값, 평균 구하기

```
SELECT
  COUNT(*) AS cnt_quantity,
  MIN(Quantity) AS min_quantity,
  MAX(Quantity) AS max_quantity,
  AVG(Quantity) AS avg_quantity
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
WHERE UnitPrice = 0;
```

[결과 이미지를 넣어주세요]

행	cnt_quantity	min_quantity	max_quantity	avg_quantity
1	33	1	12540	420.5151515151...

- UnitPrice = 0 를 제거하고 일관된 데이터셋을 유지하기

```
CREATE OR REPLACE TABLE `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data` AS
SELECT *
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
WHERE UnitPrice != 0;
```

[결과 이미지를 넣어주세요]

i 이 문으로 이름이 data인 테이블이 교체되었습니다.

11-7. RFM 스코어

Recency

- **InvoiceDate** 컬럼을 연월일 자료형으로 변경하기

```
SELECT DATE(InvoiceDate) AS InvoiceDate
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

행	InvoiceDate
1	2011-01-18
2	2011-01-18
3	2010-12-07
4	2010-12-07
5	2010-12-07
6	2010-12-07
7	2010-12-07
8	2010-12-07
9	2010-12-07
10	2010-12-07

- 가장 최근 구매 일자를 MAX() 함수로 찾아보기

```
SELECT
  MAX(DATE(InvoiceDate)) OVER() AS most_recent_date,
  DATE(InvoiceDate) AS InvoiceDay,
  *
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`;
```

[결과 이미지를 넣어주세요]

행	most_recent_date	InvoiceDay	InvoiceNo	StockCode	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Description
1	2011-12-09	2011-01-18	541431	23166	74215	2011-01-18 10:01:00 UTC	1.04	12346	United Kingdom	MEDIUM CERAMIC TOP STORA...
2	2011-12-09	2011-01-18	C541433	23166	-74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom	MEDIUM CERAMIC TOP STORA...
3	2011-12-09	2010-12-07	537626	20782	6	2010-12-07 14:57:00 UTC	5.49	12347	Iceland	CAMOUFLAGE EAR MUFF HEAD...
4	2011-12-09	2010-12-07	537626	84997B	6	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	RED 3 PIECE RETROSPOT CUTL...
5	2011-12-09	2010-12-07	537626	22774	12	2010-12-07 14:57:00 UTC	1.25	12347	Iceland	RED DRAWER KNOB ACRYLIC E...
6	2011-12-09	2010-12-07	537626	22727	4	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	ALARM CLOCK BAKELIKE RED
7	2011-12-09	2010-12-07	537626	22195	12	2010-12-07 14:57:00 UTC	1.65	12347	Iceland	LARGE HEART MEASURING SP...
8	2011-12-09	2010-12-07	537626	22772	12	2010-12-07 14:57:00 UTC	1.25	12347	Iceland	PINK DRAWER KNOB ACRYLIC E...
9	2011-12-09	2010-12-07	537626	85167B	30	2010-12-07 14:57:00 UTC	1.25	12347	Iceland	BLACK GRAND BAROQUE PHOT...
10	2011-12-09	2010-12-07	537626	22212	6	2010-12-07 14:57:00 UTC	2.1	12347	Iceland	FOUR HOOK WHITE LOVEBIRDS
11	2011-12-09	2010-12-07	537626	22497	4	2010-12-07 14:57:00 UTC	4.25	12347	Iceland	SET OF 2 TINS VINTAGE BATHR...
12	2011-12-09	2010-12-07	537626	85116	12	2010-12-07 14:57:00 UTC	2.1	12347	Iceland	BLACK CANDELABRA T-LIGHT ...
13	2011-12-09	2010-12-07	537626	21171	12	2010-12-07 14:57:00 UTC	1.45	12347	Iceland	BATHROOM METAL SIGN
14	2011-12-09	2010-12-07	537626	85232D	3	2010-12-07 14:57:00 UTC	4.95	12347	Iceland	SET/3 DECOUPAGE STACKING ...
15	2011-12-09	2010-12-07	537626	22729	4	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	ALARM CLOCK BAKELIKE ORA...

- 유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기

```

SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
GROUP BY CustomerID;

```

[결과 이미지를 넣어주세요]

행	CustomerID	InvoiceDay
1	12346	2011-01-18
2	12347	2011-12-07
3	12348	2011-09-25
4	12349	2011-11-21
5	12350	2011-02-02
6	12352	2011-11-03
7	12353	2011-05-19
8	12354	2011-04-21
9	12355	2011-05-09
10	12356	2011-11-17
11	12357	2011-11-06
12	12358	2011-12-08
13	12359	2011-12-02
14	12360	2011-10-18
15	12361	2011-02-25

- 가장 최근 일자(**most_recent_date**)와 유저별 마지막 구매일(**InvoiceDay**)간의 차이를 계산하기

```

SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
FROM (
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
  FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
  GROUP BY CustomerID
);

```

[결과 이미지를 넣어주세요]

행	CustomerID	recency
1	12386	337
2	12709	3
3	12981	29
4	13043	253
5	13216	267
6	13259	61
7	13364	66
8	13899	16
9	13911	57
10	14044	26
11	14055	106
12	14216	3
13	14235	10
14	14340	218
15	14345	38

- 최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 `user_r` 이라는 이름의 테이블로 저장하기

```

CREATE OR REPLACE TABLE `project-1fdc5a34-459d-4c61-a1b.modulabs_project.user_r` AS
SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
FROM (
  SELECT
    CustomerID,

```



```

MAX(InvoiceDate) AS InvoiceDay
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
GROUP BY CustomerID
);

```

[결과 이미지를 넣어주세요]

i 이 문으로 이름이 user_r인 새 테이블이 생성되었습니다.

행	CustomerID	recency
1	13777	0
2	12713	0
3	14422	0
4	13069	0
5	17428	0
6	12518	0
7	12985	0
8	17389	0
9	12433	0
10	16705	0

Frequency

- 고객마다 고유한 InvoiceNo의 수를 세어보기

```

SELECT
  CustomerID,
  COUNT(DISTINCT InvoiceNo) AS purchase_cnt
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
GROUP BY CustomerID;

```

[결과 이미지를 넣어주세요]

행	CustomerID	purchase_cnt
1	12346	2
2	12347	7
3	12348	4
4	12349	1
5	12350	1
6	12352	8
7	12353	1
8	12354	1
9	12355	1
10	12356	3
11	12357	1
12	12358	2
13	12359	6
14	12360	3
15	12361	1

- 각 고객 별로 구매한 아이템의 총 수량 더하기

```
SELECT
  CustomerID,
  SUM(Quantity) AS item_cnt
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
GROUP BY CustomerID;
```

[결과 이미지를 넣어주세요]

행	CustomerID	item_cnt
1	12346	0
2	12347	2458
3	12348	2332
4	12349	630
5	12350	196
6	12352	463
7	12353	20
8	12354	530
9	12355	240
10	12356	1573
11	12357	2708
12	12358	242
13	12359	1599
14	12360	1156
15	12361	90

- 전체 거래 건수 계산과 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE `project-1fdc5a34-459d-4c61-a1b.modulabs_project.user_rf` AS

-- (1) 전체 거래 건수 계산
WITH purchase_cnt AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS purchase_cnt
  FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
  GROUP BY CustomerID
),

-- (2) 구매한 아이템 총 수량 계산
item_cnt AS (
  SELECT
    CustomerID,
    SUM(Quantity) AS item_cnt
  FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
  GROUP BY CustomerID
)

-- 기존의 user_r에 (1)과 (2)를 통합
```

```

SELECT
    pc.CustomerID,
    pc.purchase_cnt,
    ic.item_cnt,
    ur.recency
FROM purchase_cnt AS pc
JOIN item_cnt AS ic
    ON pc.CustomerID = ic.CustomerID
JOIN `project-1fdc5a34-459d-4c61-a1b.modulabs_project.user_r` AS ur
    ON pc.CustomerID = ur.CustomerID;

```

[결과 이미지를 넣어주세요]

i 이 문으로 이름이 user_rf인 새 테이블이 생성되었습니다.

행	CustomerID	purchase_cnt	item_cnt	recency
1	12713	1	505	0
2	13298	1	96	1
3	15520	1	314	1
4	14569	1	79	1
5	13436	1	76	1
6	15195	1	1404	2
7	14204	1	72	2
8	15471	1	256	2
9	15992	1	17	3
10	12478	1	233	3
11	16569	1	93	3
12	14578	1	240	3
13	17914	1	457	3
14	15318	1	642	3
15	12442	1	181	3

Monetary

- 고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)

```

SELECT
    CustomerID,
    ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
GROUP BY CustomerID;

```

[결과 이미지를 넣어주세요]

행	CustomerID	user_total
1	12346	0.0
2	12347	4310.0
3	12348	1437.2
4	12349	1457.5
5	12350	294.4
6	12352	1265.4
7	12353	89.0
8	12354	1079.4
9	12355	459.4
10	12356	2487.4

- 고객별 평균 거래 금액 계산

- 고객별 평균 거래 금액을 구하기 위해 1) `data` 테이블을 `user_rf` 테이블과 조인(LEFT JOIN) 한 후, 2) `purchase_cnt` 로 나누어서 3) `user_rfm` 테이블로 저장하기

```
CREATE OR REPLACE TABLE `project-1fdc5a34-459d-4c61-a1b.modulabs_project.user_rfm` AS
SELECT
  rf.CustomerID AS CustomerID,
  rf.purchase_cnt,
  rf.item_cnt,
  rf.recency,
  ut.user_total,
  ut.user_total / rf.purchase_cnt AS user_average
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.user_rf` rf
LEFT JOIN (
  -- 고객 별 총 지출액
  SELECT
    CustomerID,
    SUM(Quantity * UnitPrice) AS user_total
  FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
  GROUP BY CustomerID
) ut
ON rf.CustomerID = ut.CustomerID;
```

[결과 이미지를 넣어주세요]

i 이 문으로 이름이 user_rfm인 새 테이블이 생성되었습니다.

RFM 통합 테이블 출력하기

- 최종 user_rfm 테이블을 출력하기

```
SELECT *
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.user_rfm`;
```

[결과 이미지를 넣어주세요]

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
1	12713	1	505	0	794.5	794.5
2	14569	1	79	1	227.4	227.4
3	15520	1	314	1	343.5	343.5
4	13436	1	76	1	196.9	196.9
5	13298	1	96	1	360.0	360.0
6	14204	1	72	2	150.6	150.6
7	15471	1	256	2	454.5	454.5
8	15195	1	1404	2	3861.0	3861.0
9	12650	1	250	3	242.4	242.4
10	15318	1	642	3	312.6	312.6
11	12442	1	181	3	144.1	144.1
12	16569	1	93	3	124.2	124.2
13	15992	1	17	3	42.0	42.0
14	14578	1	240	3	168.6	168.6
15	12478	1	233	3	546.0	546.0

11-8. 추가 Feature 추출

1. 구매하는 제품의 다양성

- 1) 고객 별로 구매한 상품들의 고유한 수를 계산하기
- 2) user_rfm 테이블과 결과를 합치기
- 3) user_data 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE `project-1fdc5a34-459d-4c61-a1b.modulabs_project.user_data` AS
WITH unique_products AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT StockCode) AS unique_products
  FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
  GROUP BY CustomerID
)
SELECT ur.*, up.* EXCEPT (CustomerID)
```

```
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.user_rfm` AS ur
JOIN unique_products AS up
ON ur.CustomerID = up.CustomerID;
```

[결과 이미지를 넣어주세요]

i 이 문으로 이름이 user_data인 새 테이블이 생성되었습니다.

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_prod...
1	14351	1	12	164	51.0	51.0	1
2	13391	1	4	203	59.8	59.8	1
3	15070	1	36	372	106.2	106.2	1
4	17923	1	50	282	207.5	207.5	1
5	13017	1	48	7	204.0	204.0	1
6	13302	1	5	155	63.8	63.8	1
7	16344	1	18	158	101.1	101.1	1
8	18233	1	4	325	440.0	440.0	1
9	17752	1	192	359	80.6	80.6	1
10	13829	1	-12	359	-102.0	-102.0	1

2. 평균 구매 주기

- 고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)
 - 균 구매 소요 일수를 계산하고, 그 결과를 **user_data**에 통합

```
CREATE OR REPLACE TABLE `project-1fdc5a34-459d-4c61-a1b.modulabs_project.u
ser_data` AS
WITH purchase_intervals AS (
  SELECT
    CustomerID,
    CASE
      WHEN ROUND(AVG(interval_), 2) IS NULL THEN 0
      ELSE ROUND(AVG(interval_), 2)
    END AS average_interval
  FROM (
    SELECT
      CustomerID,
      DATE_DIFF(
        DATE(InvoiceDate),
        LAG(DATE(InvoiceDate)) OVER (PARTITION BY CustomerID ORDER BY DATE
(InvoiceDate)),
        DAY
      ) AS interval_
    FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.data`
    WHERE CustomerID IS NOT NULL
  )
)
```

```

GROUP BY CustomerID
)

SELECT
    u.* EXCEPT (average_interval),
    pi.average_interval
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.user_data` AS u
LEFT JOIN purchase_intervals AS pi
ON u.CustomerID = pi.CustomerID;

```

[결과 이미지를 넣어주세요]

i 이 문으로 이름이 user_data인 테이블이 교체되었습니다.

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_prod...	average_inte...
1	18113	1	72	368	76.3	76.3	1	0.0
2	16257	1	1	176	21.9	21.9	1	0.0
3	12603	1	56	21	613.2	613.2	1	0.0
4	16990	1	100	218	179.0	179.0	1	0.0
5	18068	1	6	289	101.7	101.7	1	0.0
6	17715	1	384	200	326.4	326.4	1	0.0
7	14119	1	-2	354	-19.9	-19.9	1	0.0
8	16995	1	-1	372	-1.3	-1.3	1	0.0
9	18184	1	60	15	49.8	49.8	1	0.0
10	16323	1	50	196	207.5	207.5	1	0.0
11	12943	1	-1	301	-3.8	-3.8	1	0.0
12	16953	1	10	30	20.8	20.8	1	0.0
13	14576	1	12	372	35.4	35.4	1	0.0
14	16138	1	-1	368	-8.0	-8.0	1	0.0
15	17986	1	10	56	20.8	20.8	1	0.0

3. 구매 취소 경향성

- 고객의 취소 패턴 파악하기

1) 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수

2) 취소 비율(cancel_rate) : 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율

- 취소 빈도와 취소 비율을 계산하고 그 결과를 **user_data**에 통합하기
(취소 비율은 소수점 두번째 자리)

```

CREATE OR REPLACE TABLE project_name.modulabs_project.user_data AS

WITH TransactionInfo AS (
    SELECT
        CustomerID,
        # [[YOUR QUERY]] AS total_transactions,

```



```

        # [[YOUR QUERY]] AS cancel_frequency
    FROM project_name.modulabs_project.data
    # [[YOUR QUERY]]
)

SELECT u.*, t.* EXCEPT(CustomerID), # [[YOUR QUERY]] AS cancel_rate
FROM `project_name.modulabs_project.user_data` AS u
LEFT JOIN TransactionInfo AS t
ON # [[YOUR QUERY]];

```

[결과 이미지를 넣어주세요]

i 이 문으로 이름이 user_data인 테이블이 교체되었습니다.

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_prod...	average_inte...
1	18113	1	72	368	76.3	76.3	1	0.0
2	16257	1	1	176	21.9	21.9	1	0.0
3	12603	1	56	21	613.2	613.2	1	0.0
4	16990	1	100	218	179.0	179.0	1	0.0
5	18068	1	6	289	101.7	101.7	1	0.0
6	17715	1	384	200	326.4	326.4	1	0.0
7	14119	1	-2	354	-19.9	-19.9	1	0.0
8	16995	1	-1	372	-1.3	-1.3	1	0.0
9	18184	1	60	15	49.8	49.8	1	0.0
10	16323	1	50	196	207.5	207.5	1	0.0
11	12943	1	-1	301	-3.8	-3.8	1	0.0
12	16953	1	10	30	20.8	20.8	1	0.0
13	14576	1	12	372	35.4	35.4	1	0.0
14	16138	1	-1	368	-8.0	-8.0	1	0.0
15	17986	1	10	56	20.8	20.8	1	0.0

- 다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 **user_data**를 출력하기

```

SELECT *
FROM `project-1fdc5a34-459d-4c61-a1b.modulabs_project.user_data`
LIMIT 100;

```

[결과 이미지를 넣어주세요]

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	average_interval	total_transactions	cancel_frequency	cancel_rate
1	14642	1	76	58	96.1	96.1	2	0.0	1	0	0.0
2	17831	1	12	33	35.4	35.4	2	0.0	1	0	0.0
3	15049	1	63	273	121.2	121.2	3	0.0	1	0	0.0
4	16569	1	93	3	124.2	124.2	5	0.0	1	0	0.0
5	15350	1	51	373	115.7	115.7	5	0.0	1	0	0.0
6	13710	1	176	31	180.5	180.5	8	0.0	1	0	0.0
7	15419	1	230	24	135.9	135.9	8	0.0	1	0	0.0
8	13103	1	82	39	243.9	243.9	8	0.0	1	0	0.0
9	17538	1	134	25	98.4	98.4	9	0.0	1	0	0.0
10	17464	1	220	158	290.0	290.0	9	0.0	1	0	0.0
11	12938	1	140	25	114.1	114.1	9	0.0	1	0	0.0
12	16846	1	135	50	214.3	214.3	11	0.0	1	0	0.0
13	17492	1	155	248	374.6	374.6	12	0.0	1	0	0.0
14	12355	1	240	214	459.4	459.4	13	0.0	1	0	0.0
15	14411	1	196	310	1063.0	1063.0	14	0.0	1	0	0.0

회고

[회고 내용을 작성해주세요]

Keep : 데이터 전처리 과정에서 결측치 제거, 중복 제거, 이상치 처리 등을 쿼리문을 통해 단계적으로 수행하였다. 흐름이 매우 길어 중간 중간 집중력이 흐려지긴 했으나 결과적으로 같은 데이터셋이 출력되어 뿌듯하다.

Problem : 쿼리를 작성하는 과정에서 환경과 문법에 익숙하지 않아 오류가 발생하는 경우가 있었다. 많은 부분 GPT가 있었기에 해결 할 수 있었고, 테이블 구조를 명확히 파악하지 못해 중복 컬럼이나 업데이트 관련 에러가 발생하기도 했다.

Try : 앞으로는 GPT의존을 많이 줄이고 지금까지 작성한 흐름을 계속 살펴보며 데이터 흐름과 테이블 구조를 정리 하고 파악하고자 한다. 또한 아직 개념적으로 어려운 윈도우 함수, 서브쿼리 등등을 더 연습하여 복잡한 데이터에도 당황하지 않고 해결 할 수 있는 능력을 키우고 싶다.

감사합니다.

C4_조수경